

PROJET KEPLER — CIR1

Recapitulatif complet de l'interface web Three.js

Projet	Kepler — Simulation du systeme solaire
Interface	HTML5 / CSS3 / JavaScript / Three.js r128
Donnees	JSON genere par le noyau C
Rendu	WebGL via Three.js — 60 FPS
Interpolation	Hermite cubique avec vitesses physiques
Echelle	Lineaire — WORLD_SCALE = 1/1 000 000
Corps affiches	21 corps (Soleil + 8 planetes + 11 satellites + Halley)

1. Introduction et architecture

Cette partie du projet Kepler est l'interface de visualisation 3D du systeme solaire. Elle charge le fichier JSON genere par le noyau C et l'affiche en temps reel dans un canvas WebGL via Three.js. L'interface permet de naviguer librement dans le systeme solaire, de zoomer sur chaque corps celeste, et d'afficher les informations physiques en temps reel.

Le sujet demandait une visualisation avec p5.js. Nous avons choisi Three.js pour des raisons de performance — p5.js en mode WebGL ne gere pas efficacement les grandes scenes 3D et provoquait des chutes a 2-3 FPS. Three.js utilise directement le GPU via WebGL et maintient 60 FPS meme avec 6000 etoiles et 21 corps.

1.1 Architecture des fichiers

Fichier	Role	Contenu principal
config.js	Configuration	BODY_CONFIG, tailles, couleurs, masses, WORLD_SCALE
loader.js	Chargement donnees	fetch JSON, getPosition, getVelocity, checkHermite
scale.js	Conversion unites	metersToScene, kmToScene, posToVec3, formatters
interpolation.js	Lissage trajectoires	Hermite cubique + lerp adaptatif
simulation.js	Moteur Three.js	Scene, camera, LOD, trails, controles
ui.js	Interface utilisateur	Panneau lateral, selectBody, updateSidebar
style.css	Mise en page	Sidebar, boutons, sliders, theme sombre
index.html	Structure page	Canvas, panneau info, ordre de chargement scripts

1.2 Pipeline de donnees

Le pipeline complet depuis le C jusqu'a l'affichage :

```
C → trajectoire.json → fetch() → parse JSON → getInterpolatedPosition() → Three.js mesh
```

Chaque point du JSON contient $[[x,y,z], [vx,vy,vz], t]$ en metres et m/s. La conversion vers les unites Three.js se fait par $WORLD_SCALE = 1/1\,000\,000 : 1\text{ km} \rightarrow 0.000001\text{ unites Three.js}$, $1\text{ AU} \rightarrow 149.6\text{ unites Three.js}$.

2. Systeme d'echelle lineaire

2.1 Choix de l'echelle

Nous avons explore plusieurs approches pour convertir les positions en metres du JSON vers les unites Three.js :

Approche	Resultat	Probleme
1 AU = 100 unites	Systeme solaire visible	Planetes microscopiques (Terre = 0.004 unites)
Logarithmique (log10)	Planetes visibles	Orbites des satellites deformees
Exponentielle negative	Compression des grands corps	Phobos invisible
Lineaire WORLD_SCALE	Proportions reelles exactes	Retenu — LOD gere la visibilite

La solution retenue est l'echelle lineaire pure avec $WORLD_SCALE = 1/1\,000\,000$:

```
const WORLD_SCALE = 1 / 1_000_000; // 1 km → 0.000001 unites Three.js

// Resultats :
// 1 AU      = 149.6    unites Three.js
// Soleil    = 0.696    unites Three.js (rayon 696 000 km)
// Jupiter   = 0.0715   unites Three.js (rayon 71 492 km)
// Terre     = 0.00637   unites Three.js (rayon 6 371 km)
// Lune      = 0.00174   unites Three.js (rayon 1 737 km)
// Lune orbite a 0.384 unites de la Terre → 60 rayons terriens ✓
```

Pourquoi lineaire ?

C'est la seule echelle qui preserve les vraies proportions physiques. Avec une echelle logarithmique, Phobos semblerait aussi gros que la Lune. Avec l'echelle lineaire, si on zoome assez sur Mars, Phobos apparait a sa vraie distance relative.

2.2 Conversion des positions

La fonction `posToVec3` convertit une position JSON [x,y,z] en metres vers un vecteur Three.js. Le systeme de coordonnees physique (x droite, y avant, z haut) est converti vers Three.js (x droite, y haut, z vers nous) :

```
function posToVec3(pos) {
  // pos en metres depuis le JSON
  return new THREE.Vector3(
    metersToScene(pos[0]), // X physique → X Three.js
    metersToScene(pos[2]), // Z physique → Y Three.js (vertical)
    metersToScene(pos[1])  // Y physique → Z Three.js (profondeur)
  );
}
```

3. Level of Detail (LOD)

3.1 Principe

Probleme fondamental : avec l'echelle lineaire, la Terre a un rayon de 0.00637 unites Three.js. Vue de loin (radius camera = 80 unites), elle est complètement invisible. La solution est le LOD — les corps ne sont affichés que si la camera est assez proche.

La fonction `getVisibilityThreshold` calcule le seuil d'apparition de chaque corps :

```
function getVisibilityThreshold(name) {  
    const cfg = BODY_CONFIG[name];  
    if (!cfg) return 1;  
    // Visible quand la camera est a moins de 500 rayons du corps  
    return getBodySize(name) * 500;  
}
```

Exemples de seuils :

Corps	Rayon (unites)	Seuil apparition	Interpretation
Soleil	0.696	348 unites	Visible de tout le systeme solaire
Jupiter	0.0715	35.7 unites	Visible en zoom modere
Terre	0.00637	3.18 unites	Visible en zoom proche
Phobos	0.000011	0.0055 unites	Visible seulement en tres grand zoom

3.2 LOD des satellites

Les satellites posaient un probleme supplementaire : leur seuil d'apparition naturel est trop petit — ils n'apparaissaient jamais. Solution : multiplier le seuil des satellites par 50 pour qu'ils apparaissent en meme temps que leur planete parente :

```
const effective = cfg.group === 'satellite'  
    ? threshold * 50  
    : threshold;  
mesh.visible = distCam < effective;
```

Les trainers des satellites sont aussi lies a la visibilite du mesh :

```
if (trailLines[name]) {  
    trailLines[name].visible = mesh.visible;  
}
```

4. Interpolation Hermite cubique

4.1 Probleme initial

Le JSON exporte 1 point par jour par corps. A 60 FPS avec une vitesse de simulation elevee, les planetes 'se teleportent' d'un point a l'autre. Nous avons explore plusieurs solutions :

Methode tentee	Resultat	Probleme
CatmullRomCurve3 globale	Lissage apparent	Deformation des orbites, Mercure en triangle
CatmullRomCurve3 locale	Meilleur	Periode de la Lune incorrecte
Lerp lineaire	Simple	Mouvement saccade aux angles de l'orbite
Hermite cubique	Parfait	Retenu — utilise les vitesses physiques du JSON

4.2 Pourquoi Hermite avec les vitesses physiques

Le JSON contient non seulement les positions mais aussi les vitesses calculees par le moteur C. Ces vitesses sont les vraies derivees de la trajectoire physique. L'interpolation Hermite utilise exactement ces vitesses comme tangentes, garantissant :

- Passage exact par les positions JSON a chaque jour entier
- Tangentes correctes — la courbe part dans la direction de la vraie vitesse
- Pas de deformation des orbites — chaque segment est independant
- Conservation des periodes orbitales

4.3 Implementation

Les polynomes de base de Hermite h00, h10, h01, h11 :

```
function hermiteInterp(p0, v0, p1, v1, t, dt) {
    const t2 = t * t, t3 = t2 * t;
    const h00 = 2*t3 - 3*t2 + 1; // base position p0
    const h10 = t3 - 2*t2 + t; // base tangente v0
    const h01 = -2*t3 + 3*t2; // base position p1
    const h11 = t3 - t2; // base tangente v1
    // dt convertit vitesse (m/s) en deplacement sur [0,1]
    return h00*p0 + h10*dt*v0 + h01*p1 + h11*dt*v1;
}
```

Dans getInterpolatedPosition, t est la fraction entre deux jours (0 a 1) et dt=86400s :

```
const i0 = Math.floor(timeFloat);
const t = timeFloat - i0; // fraction [0, 1]
const dt = 86400; // secondes entre deux points JSON
```

4.4 Probleme de coherence Hermite — Proteus

Proteus (satellite de Neptune, periode 1.12 jours) presentait un zigzag avec l'interpolation Hermite. Diagnostic par la verification du ratio tangente/distance :

```
const ratio = (dt * ||vitesse||) / ||p1 - p0||
```

Valeurs ideales et observees :

Corps	Ratio min	Ratio max	Diagnostic
Moon	0.996	1.004	Parfait — Hermite stable
Phobos	0.845	1.155	Bon — Hermite stable
Proteus	0.305	2.888	OVERSHOOT — Hermite instable

Cause : avec SAMPLE=48 (1 point/jour), Proteus fait $1.12/1 = 1.12$ points par orbite. A certains moments les deux positions successives sont presque identiques (Proteus a fait presque exactement une orbite complete) mais les vitesses pointent en directions opposees. Hermite tente de respecter les deux tangentes sur une distance quasi nulle → overshoot massif.

Solution : lerp adaptatif — si le ratio est en dehors de [0.5, 1.8], on bascule sur une interpolation lineaire simple pour ce segment :

```
if (ratio > 1.8 || ratio < 0.5) {  
    // Lerp lineaire — Hermite instable sur ce segment  
    return new THREE.Vector3(  
        metersToScene(p0[0] + t*(p1[0]-p0[0])),  
        metersToScene(p0[2] + t*(p1[2]-p0[2])),  
        metersToScene(p0[1] + t*(p1[1]-p0[1]))  
    );  
}
```

Limite connue

Proteus utilise du lerp lineaire au lieu d'Hermite. Son mouvement est legerement moins fluide que les autres corps. La vraie solution serait d'exporter Proteus avec SAMPLE=6 (1 point toutes les 3h) pour avoir 9 points par orbite, mais cela augmente la taille du JSON.

4.5 Temps continu

Pour que l'interpolation fonctionne, le temps doit etre continu (float) et non discret (entier). timeFloat avance en jours reels a chaque frame :

```
// speed = jours par seconde de temps reel  
// 60 FPS → une frame = 1/60 seconde  
timeFloat += speed / 60.0;  
timeIndex = Math.floor(timeFloat); // pour le slider
```

5. Trainee lumineuse et orbites

5.1 Orbites statiques

Les orbites completes sont calculees une seule fois au chargement dans buildOrbitLine. Elles sont tres discrettes (opacite 8-15%) pour ne pas surcharger la scene. Les orbites des satellites sont masquees de loin et n'apparaissent qu'en zoom local.

```
function buildOrbitLine(name, cfg) {
  const pts = trajectories[name];
  const stride = cfg.group === 'satellite' ? 2 : 1;
  const points = [];
  for (let i = 0; i < pts.length; i += stride)
    points.push(posToVec3(pts[i][0]));
  points.push(points[0].clone()); // fermer la boucle
  // opacity 0.08 (satellite) ou 0.15 (planete)
}
```

5.2 Trainee dynamique

La trainee est une ligne dynamique qui suit le corps sur les derniers N jours. Elle est mise a jour a chaque frame en ajoutant la position courante et en supprimant la plus ancienne :

```
const TRAIL_LENGTH = {
  planet: 60, // 60 derniers jours
  satellite: 90, // 90 derniers jours
  comet: 120, // 120 derniers jours (Halley tres visible)
};
```

Implementation avec un buffer pre-alloue pour eviter les reallocations :

```
// Pre-allouer le buffer a la taille maximale
const positions = new Float32Array(len * 3);
geo.setAttribute('position', new THREE.BufferAttribute(positions, 3));
geo.setDrawRange(0, 0); // vide au depart

// Mise a jour a chaque frame
hist.push(pos.clone());
if (hist.length > len) hist.shift();
geo.setDrawRange(0, hist.length);
geo.attributes.position.needsUpdate = true;
```

6. Systeme de camera

6.1 Camera spherique

La camera utilise des coordonnees spheriques (theta, phi, radius) plutot que des translations cartesiennes. Ca permet une navigation naturelle autour d'une cible :

```
let spherical = { theta: 0.5, phi: 1.0, radius: 80 };

camera.position.set(
  target.x + radius * sin(phi) * sin(theta),
  target.y + radius * cos(phi),
  target.z + radius * sin(phi) * cos(theta)
);
```

La molette de la souris change uniquement radius (zoom), le drag change theta et phi (rotation). Le zoom est exponentiel pour permettre de passer de vue systeme solaire (radius=80) a vue Phobos (radius=0.001) :

```
spherical.radius *= e.deltaY > 0 ? 1.08 : 0.92;
spherical.radius = Math.max(1e-6, Math.min(5000, spherical.radius));
```

6.2 Focus sur un corps — probleme de momentum

Premier probleme : quand on selectionne un corps, la camera utilisait lerp (transition douce) vers le corps. Mais les planetes se deplacent — si le lerp est trop lent, la planete sort du champ de vision.

Deuxieme probleme : le focus se faisait derriere la planete car cameraTarget convergait avec un retard.

Solution — deux phases distinctes :

- Transition (isTransitioning=true) : lerp progressif vers le corps pour une animation fluide
- Suivi (isTransitioning=false) : copy() exacte sans lag — la camera colle au corps

```
if (isTransitioning) {
  transitionProgress += 0.05;
  cameraTarget.lerp(target, transitionProgress);
} else {
  cameraTarget.copy(target); // suivi exact, pas de lag
}
```

6.3 Zoom automatique lors de la selection

Quand on clique sur un corps dans la liste, la camera zoome automatiquement a 10 rayons du corps :

```
const size = getBodySize(name); // rayon en unites Three.js
spherical.radius = size * 10; // 10 rayons de distance
```

Exemples :

- Terre : size=0.00637 → radius=0.0637 → vue de 63.7 km au-dessus
- Jupiter : size=0.0715 → radius=0.715 → vue de 715 km au-dessus
- Soleil : size=0.696 → radius=6.96 → vue de 6960 km au-dessus

7. Problemes rencontres et solutions

7.1 Performances — ecran noir et 2 FPS

Probleme

p5.js WEBGL donnait 2-3 FPS avec 20 corps et 6000 etoiles. Les appels a sphere() pour chaque etoile etaient catastrophiques.

Solution

Migration vers Three.js. Les etoiles sont rendu avec THREE.Points (un seul draw call pour 6000 etoiles), les corps avec des SphereGeometry pre-calculées.

7.2 Variable AU declaree deux fois

Probleme

Erreur 'Identifiant AU has already been declared' au demarrage. Three.js r128 declare lui-meme une variable AU en interne.

Solution

Renommer AU en AU_METERS dans scale.js, ou plus simplement utiliser directement les valeurs numeriques.

7.3 JSON introuvable — erreur 404

Probleme

Le serveur python etait lance depuis la racine du projet, mais le chemin dans le code etait 'data/trajectoire.json'. Le JSON etait dans web/data/ donc le bon chemin depuis la racine etait 'web/data/trajectoire.json'.

Solution

Lancer le serveur depuis le dossier web/ (cd web && python -m http.server 8000) et garder le chemin 'data/trajectoire.json'.

7.4 Deuxieme Soleil fixe au centre

Probleme

Le Soleil etait cree deux fois dans buildScene : une fois manuellement (bodyMeshes['sun']) et une fois dans la boucle for sur bodyNames car le Soleil etait desormais dans le JSON.

Solution

Ajouter if (name === 'sun') continue; au debut de la boucle for.

7.5 Halo du Soleil reste a l'origine

Probleme

Le halo (sphere transparente autour du Soleil) etait ajoute a la scene avec scene.add(glowMesh) et gardait donc une position fixe a l'origine.

Solution

Ajouter le halo comme enfant du mesh du Soleil : `sunMesh.add(glowMesh)`. Les enfants Three.js suivent automatiquement leur parent.

7.6 Satellites trop gros ou invisibles selon l'echelle

Probleme

Avec une echelle logarithmique, les satellites semblaient aussi gros que les planetes. Avec l'echelle lineaire pure, ils etaient invisibles de loin.

Solution

Echelle lineaire + LOD adaptatif. Les corps n'apparaissent que si la camera est a moins de 500 rayons. Les satellites ont un seuil multiplie par 50 pour apparaitre avec leur planete.

7.7 CatmullRomCurve3 deformant les orbites

Probleme

Une spline Catmull-Rom globale construite sur toute l'orbite creait des artefacts : Mercure formait des triangles, la Lune n'effectuait plus sa revolution en 27 jours.

Cause

La spline globale interpole entre tous les points en essayant de creer une courbe 'douce', ce qui peut modifier localement la forme de l'orbite et la vitesse angulaire.

Solution

Interpolation Hermite cubique locale — chaque segment entre deux points consecutifs est independant et utilise les vraies vitesses physiques du JSON comme tangentes.

7.8 Proteus — zigzag avec Hermite

Probleme

Proteus (periode 1.12 jours) presentait un zigzag localement sur son orbite. Ratio Hermite : min 0.305, max 2.888 — tres eloigne de 1.

Cause

Avec SAMPLE=48 (1 point/jour), Proteus fait presque exactement 1 orbite complete entre deux points exportes. A certains moments les positions sont quasi identiques mais les vitesses opposees → Hermite overshoot.

Solution

Lerp adaptatif : si le ratio $(dt * ||v||) / ||p1 - p0||$ est hors de [0.5, 1.8], on utilise un lerp lineaire simple pour ce segment. Proteus est legerement moins fluide mais sans artefact.

8. Fonctionnalités implémentées

Fonctionnalité	Statut
Rendu 3D WebGL via Three.js	Termine
Chargement JSON avec fetch()	Termine
21 corps célestes animés	Termine
Echelle linéaire réelle WORLD_SCALE	Termine
Level of Detail (LOD) adaptatif	Termine
Interpolation Hermite cubique	Termine
Lerp adaptatif pour Proteus	Termine
Trainée lumineuse dynamique	Termine
Orbites statiques discrètes	Termine
6000 étoiles de fond (GPU)	Termine
Rotation + zoom souris	Termine
Focus avec suivi exact (sans lag)	Termine
Panneau latéral — énergie, vitesse, distance	Termine
Marqueur barycentre (croix grise)	Termine
Animation play/pause + slider vitesse	Termine
Slider temps + temps simulé affiche	Termine
Liste corps cliquable dans la sidebar	Termine
Halo lumineux du Soleil	Termine
Déplacement du Soleil (barycentre)	Termine

9. Sources et références

- Three.js r128 — <https://threejs.org/docs/>
- THREE.CatmullRomCurve3 — abandonnée au profit de Hermite
- Interpolation de Hermite — Wikipedia 'Cubic Hermite spline'
- NASA Eyes on the Solar System — référence pour le rendu LOD
- MDN Web Docs — fetch API, Float32Array, BufferGeometry
- Three.js BufferGeometry — optimisation des lignes dynamiques
- Tamayo et al. (2019) REBOUND — même principe d'interpolation physique

Conseil pour l'oral

La migration de p5.js vers Three.js est une décision technique justifiable : p5.js est une bibliothèque pédagogique, Three.js est une bibliothèque de production. Pour une simulation scientifique 3D avec des dizaines de corps et des milliers d'étoiles, Three.js est le choix professionnel. NASA Eyes on the Solar System utilise d'ailleurs WebGL directement.